

Iceberg Optimizer Skill

Benchmark Report — Version 4

June 2026

Branch	claude/iceberg-optimization-research-anmbwq
Skill	skills/iceberg-optimizer/
Harness	tests/skill_benchmark/run_benchmark.py
Model	Claude (claude CLI, default model)
Scenarios	22 (expanded from 5 in v1/v2)
Overall (skill)	PASS — 22/22 scenarios, avg judge score 5.0/5
Baseline comparison	8 scenarios; skill lifts 3 of them

Contents

1	Executive Summary	3
2	Skill Architecture	3
2.1	Design Principle	3
2.2	Three-Mode Execution Model	3
2.3	Phase Flow	4
2.4	Candidate Actions	4
3	Benchmark Methodology	5
3.1	Approach	5
3.2	Scenario Categories	5
4	Scenario Descriptions	6
4.1	Original Five Scenarios (from v1/v2)	6
4.1.1	cold_archive	6
4.1.2	streaming_thin_spread	6
4.1.3	gdpr_deletes	6
4.1.4	partition_misalignment	6
4.1.5	snapshot_bloat_only	6
4.2	Seventeen New Scenarios (v3)	6
4.2.1	position_delete_accumulation	6
4.2.2	format_version_mismatch	6
4.2.3	over_partitioned_tiny_partitions	6
4.2.4	flink_micro_commit_scatter	7
4.2.5	late_arriving_data	7
4.2.6	wide_table_memory_pressure	7
4.2.7	cdc_high_churn_cow_consideration	7
4.2.8	query_cost_vs_maintenance_cost	7
4.2.9	snapshot_time_travel_cdc	7
4.2.10	mixed_partition_spec	7
4.2.11	bloom_filter_high_cardinality	7
4.2.12	gdpr_ordering_mistake	7
4.2.13	z_order_too_many_columns	8
4.2.14	hot_partition_conflict	8
4.2.15	orphan_files_before_expiry	8
4.2.16	bloom_filter_wrong_column	8
4.2.17	streaming_death_spiral	8
5	Results	8
5.1	Summary Table	8
5.2	Score Distribution	10
5.3	Historical Score Progression	10
5.4	Baseline Comparison	10
6	Key Scenario Deep Dives	11
6.1	Format Version Mismatch: L Is a Prerequisite, Not an Option	11
6.2	Z-Order Too Many Columns: Dimensional Collapse	12
6.3	Snapshot Time Travel CDC: Time-Based vs Count-Based Expiry	12
6.4	Streaming Death Spiral: Write-Time Fix Must Come First	12
6.5	Hot Partition Conflict: Never Compact the Active Partition	13

6.6	Orphan Files Before Expiry: Ordering Matters	13
7	Skill Review	13
7.1	Architecture Improvements	13
7.2	Strengths	14
7.3	Residual Improvement Areas	14
8	Benchmark Infrastructure	14
8.1	File Structure	14
8.2	Usage	15
9	Conclusion	15

1 Executive Summary

The **iceberg-optimizer** skill is a Claude Code skill that diagnoses Apache Iceberg tables and produces a ranked, cost-aware maintenance plan. This report documents the fourth benchmark run: 22 scenarios with a full skill vs. baseline comparison on 8 of them.

Previous runs (v1/v2) achieved 4.8/5 and 5.0/5 on the original five scenarios. Version 3 expanded coverage to 22 scenarios (avg 4.86/5) and identified two below-5 cases: **flink_micro_commit_scatter** (3/5 — z-order erroneously included) and **late_arriving_data** (4/5 — K recommended despite already being set). Version 4 fixes both via targeted improvements to the decision framework and fixture data, achieving a **perfect 5.0/5** across all 22 scenarios.

Version 4 also introduces a **baseline comparison** mode (`--no-skill` flag) that sends only raw table metadata and interview answers — no skill document, no workload analysis, no cost simulation output. This isolates what the skill framework actually adds. Key finding: the skill’s value is concentrated in *cross-signal reasoning* (profile $\times$ workload signals $\times$ user intent $\rightarrow$ action selection), not in general Iceberg domain knowledge which Claude already has.

Anti-patterns sourced from:

- *Architecting Apache Iceberg Lakehouse* (O’Reilly, 2024)
- Streaming and CDC production failure patterns
- Correctness-critical scenarios: wrong maintenance ordering, incorrect bloom filter configuration, z-order dimensional collapse
- Cost vs maintenance trade-off scenarios

The 22 scenarios probe edge cases the original five did not cover: format version prerequisites, delete-file type discrimination (E1 vs E2), time-based vs count-based snapshot expiry, high-dimensional z-order degradation, hot-partition concurrency conflicts, and the streaming death spiral.

2 Skill Architecture

2.1 Design Principle

The skill follows one governing rule: *observe before you ask, ask before you decide, simulate before you recommend*. It reads everything it can from table metadata and query logs before asking the user a single question, then only asks about intent that metadata cannot reveal.

2.2 Three-Mode Execution Model

Phase 0 detects one of three execution modes before any other work begins:

Direct Engine CLI (`trino`, `spark-sql`, `beeline`) or environment variables (`TRINO_URL`, `SPARK_HOME`, `DATABRICKS_HOST`) are present in the environment. The skill connects directly and runs profiling scripts without user intervention.

Ask-User No live connection detected. The skill guides the user through exporting the required files and then processes them.

Exported Pre-exported snapshot bundle is provided. The skill reads the bundle and proceeds without running scripts live.

Reference files are loaded *gradually*: only the sections relevant to the current phase are loaded. At Phase 5, `procedures.md` is grepped for only the matching engine and selected actions, keeping the context window narrow.

2.3 Phase Flow

Table 1: Iceberg optimizer phase flow

Phase	Name	Purpose
0	Scope & Safety	Identify table, engine, catalog; detect execution mode; read-only until Phase 5.
1	Profile	Run <code>profile_table.py</code> : file sizes, delete pressure, snapshot bloat, partition fan-out.
2a	Workload Derive	Extract write cadence, small-file patterns, delete accumulation, and query filter columns from metadata logs.
2b	Workload Interview	Ask only the unknowable intent questions: latency, frequency, cost priority, time-travel, compliance.
3	Decide	Apply the decision framework: trigger + gate evaluation $\rightarrow$ ranked candidate actions (including “do nothing”).
4	Simulate	Run <code>simulate.py</code> : Do-nothing / Light / Targeted-sort / Aggressive / Storage-min on four cost axes.
5	Plan	Emit exact, engine-specific SQL commands + maintenance schedule + monitoring thresholds.

2.4 Candidate Actions

The decision framework defines thirteen candidate actions (A–L and Z):

- A** Bin-pack compaction — resize small files to target.
- B** Sort compaction — cluster by a single leading key.
- C** Z-order compaction — multi-dimensional clustering (2–4 high-cardinality cols *maximum*).
- D** Partition evolution — zero-downtime metadata-only re-partitioning.
- E1** Equality-delete compaction — physically remove logically-deleted rows; GDPR compliance bypass gate.
- E2** Position-delete compaction — merge position deletes; lower urgency than E1.
- F** Snapshot expiry — always include; size to intent (time-based for high-churn CDC tables).
- G** Manifest rewrite / clustering — consolidate manifests; after data rewrites.
- H** Orphan-file removal — destructive; dry-run first; always after snapshot expiry.
- I** Bloom filters — high-cardinality equality lookups only (not range predicates).
- J** Write-time tuning — fix ingestion distribution and file size before compacting its output.
- K** Write-time sort order — free clustering for future writes; no rewrite cost.
- L** Format-version upgrade — one-time prerequisite for row-level delete workloads (v1 $\rightarrow$ v2).
- Z** Do nothing / minimal — explicit outcome for cold or rarely-queried tables.

3 Benchmark Methodology

3.1 Approach

A *skill-level* benchmark evaluates the full end-to-end recommendation rather than any individual script. It answers: *given a table’s complete profile and scripted interview answers, does the skill give the right advice?*

The harness (`run_benchmark.py`) supports two modes:

Skill mode (default).

1. Load `SKILL.md` and all `references/*.md` files as the system prompt — the same context Claude receives when the skill is invoked.
2. For each scenario, build a single-turn user message containing:
 - Pre-computed `profile.json` (from `profile_table.py`).
 - Pre-computed `workload.json` (from `parse_query_log.py`).
 - Pre-computed simulator output (from `simulate.py`).
 - Scripted Phase 2b interview answers from `scenarios.json`.
3. Call the `claude -p` CLI in non-interactive mode (no tool use, pure text response).
4. Run **LLM-as-judge**: a separate Claude invocation is given the scenario’s `expected_outcome` description and the skill’s response; it returns a score 1–5 and a binary PASS/FAIL (threshold: score ≥ 3).

Baseline mode (`--no-skill`). Identical to skill mode except:

- The system prompt is a minimal 247-character generic expert prompt (“*You are an expert Apache Iceberg data engineer...* ”) — no `SKILL.md`, no reference documents.
- The user message contains *only* `profile.json` and the scripted Phase 2b interview answers — no `workload.json` and no simulator output.

This isolates the contribution of the skill framework: workload signal interpretation, cost simulation, and the decision-framework’s cross-signal reasoning. The baseline agent must derive everything from raw table metadata and stated user priorities alone.

The judge evaluates correctness of reasoning rather than surface-level term presence. Each scenario’s `expected_outcome` is written in plain English so the judge can assess whether the skill understood the problem, not just whether it produced specific keywords.

3.2 Scenario Categories

The 22 scenarios fall into six categories:

Basic maintenance `cold_archive`, `snapshot_bloat_only`, `query_cost_vs_maintenance_cost` — do-nothing / minimal scenarios.

Compaction strategy `streaming_thin_spread`, `flink_micro_commit_scatter`, `streaming_death_spiral`, `late_arriving_data`, `position_delete_accumulation`, `cdc_high_churn_cow_consideration`, `wide_table_memory_p`

Partitioning `partition_misalignment`, `over_partitioned_tiny_partitions`, `mixed_partition_spec`, `hot_partition_co`

Delete management `gdpr_deletes`, `gdpr_ordering_mistake`, `format_version_mismatch`, `snapshot_time_travel_co`

Advanced optimisation `z_order_too_many_columns`, `bloom_filter_high_cardinality`, `bloom_filter_wrong_colum`

Maintenance ordering orphan_files_before_expiry, gdpr_ordering_mistake.

4 Scenario Descriptions

4.1 Original Five Scenarios (from v1/v2)

4.1.1 cold_archive

Cold archive with 10×300 MB files, 3–5 queries/year, no delete pressure. **Correct answer:** Z (do nothing) or F (snapshot expiry only). Maintenance would be net-negative at this query rate.

4.1.2 streaming_thin_spread

Streaming table: 5000 files $\times$ 1 MB (30-second commits, 20 partitions/commit). Interactive dashboards on `tenant_id + event_time`. **Correct answer:** A (bin-pack cold partitions) + C (z-order `tenant_id, event_time`) + J (`distribution-mode=hash`) + F.

4.1.3 gdpr_deletes

GDPR table: 10% equality-delete pressure, daily purges. **Correct answer:** E1 (`rewrite_data_files + remove-dangling-deletes`) then F (`retain_last=1`). Must warn that snapshot history is a GDPR liability.

4.1.4 partition_misalignment

Healthy profile (`partition_prune_rate=0.0`) but every query full-scans 12.5 GB because the partition key (`event_date`) does not match the dominant filter (`tenant_id`). **Correct answer:** D (partition evolution to bucket by `tenant_id`) or B/C.

4.1.5 snapshot_bloat_only

5×256 MB perfectly-sized files, 1 200 hourly snapshots. **Correct answer:** F only — no data file compaction (files at target).

4.2 Seventeen New Scenarios (v3)

4.2.1 position_delete_accumulation

MOR table: 240 position-delete files against 200 data files (15% position-delete pressure, 280 merge commits). **Correct answer:** E2 (`rewrite_position_delete_files`, NOT E1) + conservative F (retain ≥ 7 days). Mention COW as a long-term option.

4.2.2 format_version_mismatch

v1 Iceberg table with accumulating equality deletes from a CDC pipeline. **Correct answer:** L first (`ALTER TABLE SET 'format-version'='2'`) then E1. A v1 table cannot safely compact equality delete files.

4.2.3 over_partitioned_tiny_partitions

5-level partition spec (year/month/day/hour/region): $\approx 219\,000$ partitions, 8 MB avg files, 437k total files. **Correct answer:** D (partition evolution, drop hour + day dimensions). Bin-pack alone is a band-aid.

4.2.4 flink_micro_commit_scatter

Flink streaming: 0.5 MB files scattered across 50 partitions per 30-second commit (extreme thin spread). Queried by `event_type + region`. **Correct answer:** J (`distribution-mode=hash` on `event_type`) + K (write-time sort) + A (bin-pack cold partitions) + G. *Must NOT recommend z-order.*

4.2.5 late_arriving_data

Batch ETL: write-time sort order already set (`event_date, customer_id`) but 5% of events arrive 7–30 days late, re-opening old partitions. **Correct answer:** B (sort compaction on recently-modified partitions). *Must NOT recommend adding K again — it is already set.*

4.2.6 wide_table_memory_pressure

250-column table with 512 MB target file size (default). Compaction OOMs. Files are at target — but the target itself is wrong for wide schemas. **Correct answer:** Reduce `write.target-file-size-bytes` to 128–256 MB, then sort compaction.

4.2.7 cdc_high_churn_cow_consideration

MOR table: 8% equality-delete pressure, 12 MB avg write files (well below 256 MB target), 1 000 queries/month, interactive latency required. **Correct answer:** E1 + J (fix 12 MB small writes) + A + COW evaluation.

4.2.8 query_cost_vs_maintenance_cost

Cold archive: 3 queries/year, 512 MB files, no deletes. Compaction would cost \$200; lifetime query savings ≈\$100. **Correct answer:** Z explicit do-nothing. Only periodic F justified.

4.2.9 snapshot_time_travel_cdc

CDC table at 1 commit/minute, retention policy requires 30 days rollback window. Currently using `retain_last=100` (= 100 minutes!). **Correct answer:** F with `max-snapshot-age-ms=2592000000` (30 days). *Must NOT use count-based `retain_last` for high-frequency tables.*

4.2.10 mixed_partition_spec

Two simultaneous partition specs: old (year, month) covers 600 GB, new (year, month, bucket(10, tenant_id)) covers 200 GB. Planning overhead severe. **Correct answer:** D + full data rewrite of old-spec 600 GB. Manifest rewrite (G) alone does not move data files.

4.2.11 bloom_filter_high_cardinality

Table queried primarily by `product_id` equality (10M distinct values). No bloom filter. Min/max useless at 10M cardinality. **Correct answer:** I (enable bloom filter on `product_id`).

4.2.12 gdpr_ordering_mistake

Snapshots expired before equality-delete compaction ran (wrong order). Deleted rows may still be physically present. **Correct answer:** Urgent E1 NOW. Must explain correct order: compact THEN expire.

4.2.13 z_order_too_many_columns

Z-order on 6 columns (`tenant_id`, `event_time`, `region`, `product_id`, `channel`, `device_type`). Z-curve locality collapses beyond 4–5 dimensions. **Correct answer:** Replace with 2-column z-order (`tenant_id`, `event_time`) or plain sort.

4.2.14 hot_partition_conflict

Hourly compaction targeting today’s active partition while Flink writes to it: 80% conflict/abort rate. **Correct answer:** Add `WHERE event_date < current_date()` to compaction — target only closed (cold) partitions.

4.2.15 orphan_files_before_expiry

Orphan file removal scheduled *before* snapshot expiry. **Correct answer:** Always `expire_snapshots` first, then `remove_orphan_files` with `dry_run=true` + ≥ 3 -day safety margin.

4.2.16 bloom_filter_wrong_column

Bloom filter on `event_date` (365 distinct values, range predicates). Bloom filters are useless for ranges and wasteful for low-cardinality columns. **Correct answer:** Remove the bloom filter from `event_date`.

4.2.17 streaming_death_spiral

EMERGENCY: 180k files, 1.2 MB avg, 500 MB/min ingest rate; nightly compaction already taking 8 hours on a 2 TB table. Compaction cannot keep up. **Correct answer:** Fix write-time FIRST (`distribution-mode=hash` + `target-file-size-bytes=256MB`), then emergency bin-pack on cold partitions only. Without write-time fix, compaction is on a treadmill.

5 Results

5.1 Summary Table

Scenario	Score	Key judge finding
cold_archive	5 / 5	Correctly selects Action Z + snapshot expiry; quantifies why maintenance is net-negative at 3–5 queries/year.
streaming_thin_spread	5 / 5	Covers all four required actions in correct order: bin-pack, z-order on <code>tenant_id</code> + <code>event_time</code> , <code>distribution-mode=hash</code> , aggressive snapshot expiry.
gdpr_deletes	5 / 5	Mandates <code>rewrite_data_files(remove-dangling-deletes=true)</code> then <code>expire_snapshots</code> ; explicitly warns that retained snapshots are a GDPR liability.
partition_misalignment	5 / 5	Correctly identifies partition-column mismatch; promotes partition evolution (Action D) as primary fix due to <code>prune_rate=0.0</code> .
snapshot_bloat_only	5 / 5	Snapshot expiry identified as the only required action; data file compaction explicitly excluded.

Scenario	Score	Key judge finding
position_delete_accumulation	5 / 5	Correctly selects E2 (<code>rewrite_position_delete_files</code>), not E1; conservative expiry (retain ≥ 7 days); mentions COW consideration.
format_version_mismatch	5 / 5	Identifies <code>format_version=1</code> and mandates Action L (upgrade to v2) as Step 0 before any E1 compaction.
over_partitioned_tiny_partitions	5 / 5	Diagnoses 5-level over-partition spec (219k partitions); prescribes partition evolution to reduce granularity; correctly rejects bin-pack as primary fix.
flink_micro_commit_scatter	5 / 5	Identifies root cause as writer-side thin-spread; prescribes <code>distribution-mode=hash</code> (J) first, then write-time sort on (<code>event_type, region</code>) (K), bin-pack on cold partitions, manifest rewrite (G); correctly excludes z-order (all filters are equality-only).
late_arriving_data	5 / 5	Correctly prescribes sort compaction (B) on recently-modified partitions in a 7–30-day window; explicitly notes that write-time sort order (K) is already configured and must not be added again; does not recommend CTAS.
wide_table_memory_pressure	5 / 5	Identifies 512 MB target as wrong for a 250-column wide table; prescribes reducing <code>write.target-file-size-bytes</code> to 128–256 MB as the primary fix.
cdc_high_churn_cow_consideration	5 / 5	Covers all four required elements: E1 (equality-delete compaction), J (write-time fix for 12 MB files), A (bin-pack), and COW mode evaluation.
query_cost_vs_maintenance_cost	5 / 5	Selects explicit Action Z (do nothing); correctly quantifies that maintenance cost exceeds lifetime query savings at 3 queries/year.
snapshot_time_travel_cdc	5 / 5	Identifies that <code>retain_last=100</code> gives only 100 minutes at 1 commit/min; prescribes time-based <code>max-snapshot-age-ms=2592000000</code> (30 days).
mixed_partition_spec	5 / 5	Correctly identifies mixed-spec problem; prescribes physical data rewrite of old-spec data, not manifest rewrite alone.
bloom_filter_high_cardinality	5 / 5	Prescribes bloom filter on <code>product_id</code> (10M distinct values); explains why min/max statistics are useless for high-cardinality equality lookups.
gdpr_ordering_mistake	5 / 5	Treats E1 as urgent immediate action; explains correct order (compact THEN expire) and why the current state is non-compliant.

Scenario	Score	Key judge finding
z_order_too_many_columns	5 / 5	Identifies 6-column Z-curve dimensional collapse; prescribes reducing to 2-column z-order on (<code>tenant_id</code> , <code>event_time</code>).
hot_partition_conflict	5 / 5	Identifies hot-partition conflict root cause; prescribes <code>WHERE event_date < current_date()</code> to target only cold partitions.
orphan_files_before_expiry	5 / 5	Identifies maintenance ordering violation; prescribes expire-first then orphan-cleanup with dry-run and ≥ 3 -day safety margin.
bloom_filter_wrong_column	5 / 5	Identifies bloom filter on <code>event_date</code> (365 distinct values, range predicates) as misconfigured; prescribes removal.
streaming_death_spiral	5 / 5	Treats as emergency; explicitly mandates write-time fix (<code>distribution-mode=hash</code>) first; explains compaction alone cannot break the spiral.
Total	5.0 / 5	22/22 scenarios PASS (100%)

5.2 Score Distribution

$$\bar{s} = \frac{22 \times 5}{22} = 5.0$$

Table 3: Score distribution across 22 scenarios

Score	Count	Scenarios
5/5	22	All
Pass ($\geq 3/5$)	22	100%

5.3 Historical Score Progression

Table 4: Benchmark version history

Version	Date	Scenarios	Avg Score	Pass Rate
v1	Jun 2026	5	4.8/5	4/5 (80%)
v2	Jun 2026	5	5.0/5	5/5 (100%)
v3	Jun 2026	22	4.86/5	22/22 (100%)
v4	Jun 2026	22	5.0/5	22/22 (100%) + baseline comparison

5.4 Baseline Comparison

The baseline agent receives only `profile.json` and interview answers — no workload analysis, no cost simulation, no skill document. The comparison was run on 8 representative scenarios.

Deep Dive: `late_arriving_data` (baseline 2/5 FAIL)

The `late_arriving_data` scenario is the clearest demonstration of what the skill framework adds. Without workload signals, the baseline agent:

Table 5: Skill vs. baseline comparison (8 scenarios, LLM-judge scores)

Scenario	Baseline	Skill	Δ	Baseline failure mode
format_version_mismatch	5/5	5/5	0	—
position_delete_accumulation	5/5	5/5	0	—
gdpr_deletes	5/5	5/5	0	Missing keyword only*
snapshot_time_travel_cdc	5/5	5/5	0	—
z_order_too_many_columns	5/5	5/5	0	—
cdc_high_churn_cow_consider.	4/5	5/5	+1	Did not surface COW mode evaluation
flink_micro_commit_scatter	4/5	5/5	+1	Included z-order (no filter-type reasoning)
late_arriving_data	2/5 FAIL	5/5	+3	(see below)
Average	4.38/5	5.0/5	+0.63	

* LLM judge passed (5/5); keyword check failed on `remove-dangling-deletes`.

- Recommended *bin-pack* compaction (Action A) instead of sort compaction (Action B) on recently-modified partitions.
- Recommended rebuilding the sort key via CTAS (Action K analogue), despite the table already having `has_sort_order=true`.
- Incorrectly described the existing sort order as `event_id ASC` instead of reading it from the profile (`event_date`, `customer_id`).
- Did not scope compaction to the 7–30-day late-arrival window — it recommended full-table compaction.

The skill fixes all four failures because it reads `has_sort_order=true` from `profile.json` (skipping K), reads the `late_data` flag from `workload.json` (triggering B on affected partitions only), and applies the K prerequisite check from `decision-framework.md`.

Interpretation

The skill’s largest lift occurs where **cross-signal reasoning** is required: combining profile metadata + workload signals + user intent to select the correct action among multiple plausible alternatives.

For scenarios where a single strong signal dominates (format version mismatch, GDPR ordering, time math error, z-order collapse), the baseline already scores 5/5 — the knowledge is in Claude’s training data and raw metadata is sufficient.

For scenarios where the correct action depends on combining *multiple weak signals* (late arrival + existing sort order + affected partitions only; Flink thin spread + equality-only predicates; CDC + 12 MB writes + COW evaluation), the skill framework provides the structured reasoning path that the baseline misses.

6 Key Scenario Deep Dives

6.1 Format Version Mismatch: L Is a Prerequisite, Not an Option

This scenario tests a prerequisite ordering that is easy to miss: you cannot safely run equality-delete compaction (E1) on a v1 Iceberg table. Iceberg v1 does not have a stable specification for equality-delete files, so the `rewrite_data_files` procedure may produce undefined behavior. The mandatory first step is:

```
ALTER TABLE catalog.schema.orders
SET TBLPROPERTIES ('format-version' = '2');
```

Only after this upgrade is E1 compaction safe. The skill must identify the `format_version = 1` signal in the profile and surface L as a blocking prerequisite.

6.2 Z-Order Too Many Columns: Dimensional Collapse

Z-order interleaves bits across all specified columns to create a single space-filling curve. In 2–4 dimensions, this provides meaningful locality. In 6 dimensions, the Z-curve visits every region of the 6D hypercube in rapid succession — all file ranges overlap and data skipping fails entirely.

The paradox: adding more columns to z-order makes it *worse*. The correct fix is to reduce to the 2 most-queried columns (`tenant_id`, `event_time`).

6.3 Snapshot Time Travel CDC: Time-Based vs Count-Based Expiry

With 1 commit per minute, the math is stark:

$$\text{retain_last} = 100 \implies \text{window} = 100 \text{ minutes} = 1.7 \text{ hours}$$

$$\text{required window} = 30 \text{ days} = 43\,200 \text{ minutes}$$

The correct setting is:

```
CALL catalog.system.expire_snapshots(
  table => 'prod.cdc.transaction_log',
  older_than => TIMESTAMP '...', -- or use max-snapshot-age-ms property
  retain_last => 1
);
-- Table property:
ALTER TABLE ... SET TBLPROPERTIES (
  'history.expire.max-snapshot-age-ms' = '259200000' -- 30 days
);
```

Count-based `retain_last` is dangerous for high-frequency commit tables. The skill must detect the 1-commit/minute cadence and flag this.

6.4 Streaming Death Spiral: Write-Time Fix Must Come First

This scenario tests whether the skill understands that compaction is a palliative, not a cure, when the ingest rate exceeds the compaction throughput:

$$\text{ingest rate} = 500 \text{ MB/min} = 30 \text{ GB/hr}$$

$$\text{compaction throughput} = \frac{2 \text{ TB}}{8 \text{ hr}} = 256 \text{ GB/hr}$$

At first glance compaction has headroom, but each hour of ingest creates $30 \text{ GB} / 1.2 \text{ MB} = 25\,000$ new small files. Compaction spends most of its time on I/O per small file, so effective compaction throughput for small files is a fraction of the headline number.

The write-time fix eliminates future small-file creation. Without it, compaction perpetually falls further behind.

6.5 Hot Partition Conflict: Never Compact the Active Partition

Streaming writes and compaction on the same partition cause optimistic concurrency conflicts because Iceberg guarantees snapshot isolation: both operations try to replace the same snapshot, but only one can win.

The fix is a single `WHERE` clause:

```
CALL catalog.system.rewrite_data_files(
  table => 'prod.streaming.sensor_readings',
  where => 'event_date < current_date()'
);
```

This restricts compaction to yesterday-and-older partitions where writing has already stopped.

6.6 Orphan Files Before Expiry: Ordering Matters

Incorrect order:

1. `remove_orphan_files` — misses files still referenced by to-be-expired snapshots; later, expired snapshots drop those references and the files become true orphans *that were already skipped*.

Correct order:

1. `expire_snapshots` — dereferences files that were referenced only by expired snapshots.
2. `remove_orphan_files --dry-run` with ≥ 3 -day lag — safely removes files with no live snapshot reference.

7 Skill Review

7.1 Architecture Improvements

v3 improvements (from v2).

1. **Three-mode detection (SKILL.md).** Phase 0 explicitly detects Direct / Ask-User / Exported execution mode and loads only the reference sections needed for the detected mode.
2. **Book-derived additions (12 gaps).** From *Architecting Apache Iceberg Lakehouse*: v3 deletion vectors, `rewrite_position_delete_files`, file-sizing guidance for wide tables (128–256 MB for 250+ columns), CDC time-based snapshot expiry, conditional compaction, delete-file sawtooth diagnostic, format version migration, write-time sort order (Action K), and more.
3. **17 new benchmark scenarios.** Action discrimination (E1 vs E2), format version prerequisites (L), over-partitioning, Flink scatter, late-arriving data, wide-table OOM, CDC COW evaluation, cost vs maintenance, time-based expiry, mixed partition specs, bloom filter placement, GDPR ordering, z-order collapse, hot partition conflict, orphan ordering, streaming death spiral.

v4 improvements (this report). Two scenarios scored below 5/5 in v3. Both were fixed via targeted, generalised improvements to the decision framework — no scenario-specific overrides:

1. **Z-order predicate-type rule (flink 3/5 → 5/5).** Action C in `decision-framework.md` now states: “If ALL filter columns use equality predicates only (no range column), do NOT recommend Z-order — choose Sort (B) or write-time sort order (K) instead. Z-order’s

extra cost is only justified when range predicates must be combined with equality filters across different dimensions.” The Flink fixture’s `workload.json` was also corrected: the original file listed `tenant_id` (equality) + `event_time` (range) — a mixed pattern that correctly triggers z-order — when the scenario intent was `event_type` + `region`, both equality.

2. **K prerequisite check (late_arriving 4/5 → 5/5).** Action K in `decision-framework.md` now has an explicit prerequisite: *“Before evaluating K, read `has_sort_order` from the profile. If `has_sort_order=true`, skip K entirely and note: ‘Write-time sort order is already configured — no change needed.’”* The trigger was also broadened to apply to both range and equality filter columns, since sorting on a low-cardinality equality column (e.g. `event_type`) still groups matching rows into contiguous file ranges.

7.2 Strengths

- **Trigger/gate separation.** Metadata triggers and intent gates are cleanly distinguished.
- **“Do nothing” as a first-class outcome.** Action Z is documented with the same depth as aggressive compaction.
- **Compliance path.** GDPR sequence is correct, ordered, and bypasses the query-frequency gate.
- **Derive-first interview.** Phase 2a derives all answerable signals before asking the user anything.
- **Write-time fix priority.** Decision framework ranks J above A/B/C so the root cause (bad ingest) is fixed before treating symptoms.
- **Action discrimination.** E1 vs E2 is clearly separated; L is a blocking prerequisite for E1 on v1 tables; H is always after F.

7.3 Residual Improvement Areas

1. **[Medium] No-query-log fallback.** Document the cold-start path when neither Spark event log nor Trino query log is available; the skill currently assumes workload data exists.
2. **[Low] metadata-tables.md prioritisation.** The reference is thorough but unprioritised — adding a “Quick Diagnostics” summary section would improve Phase 1 efficiency.
3. **[Low] Expand baseline comparison to all 22 scenarios.** The 8- scenario sample already shows the pattern clearly, but a full 22-scenario baseline run would give a definitive lift measurement.

8 Benchmark Infrastructure

8.1 File Structure

```
skills/iceberg-optimizer/
  SKILL.md
  references/
    decision-framework.md  metadata-tables.md  procedures.md
    scheduling.md           workload-interview.md testing.md
  scripts/
    profile_table.py        parse_query_log.py   simulate.py
  tests/
    test_profiler.py        test_query_log.py
  skill_benchmark/
    run_benchmark.py        # harness (claude CLI, retry, LLM judge)
```

```

scenarios.json          # 22 scenarios with expected_outcome
fixtures/
  cold_archive/
  streaming_thin_spread/
  gdpr_deletes/
  partition_misalignment/
  snapshot_bloat_only/
  position_delete_accumulation/
  format_version_mismatch/
  over_partitioned_tiny_partitions/
  flink_micro_commit_scatter/
  late_arriving_data/
  wide_table_memory_pressure/
  cdc_high_churn_cow_consideration/
  query_cost_vs_maintenance_cost/
  snapshot_time_travel_cdc/
  mixed_partition_spec/
  bloom_filter_high_cardinality/
  gdpr_ordering_mistake/
  z_order_too_many_columns/
  hot_partition_conflict/
  orphan_files_before_expiry/
  bloom_filter_wrong_column/
  streaming_death_spiral/

```

8.2 Usage

```

cd skills/iceberg-optimizer/tests/skill_benchmark

# Single scenario with verbose output
python run_benchmark.py --scenario gdpr_deletes --judge --verbose

# Full suite (skill mode)
python run_benchmark.py --all --judge --output-json results.json

# Baseline comparison (profile.json + interview answers only; no skill)
python run_benchmark.py --scenario late_arriving_data --judge --no-skill
python run_benchmark.py --all --judge --no-skill --output-json baseline.json

# Individual scenarios
python run_benchmark.py --scenario streaming_death_spiral --judge
python run_benchmark.py --scenario hot_partition_conflict --judge
python run_benchmark.py --scenario orphan_files_before_expiry --judge

```

9 Conclusion

Version 4 of the iceberg-optimizer benchmark achieves a **perfect 5.0/5** across all 22 scenarios (up from 4.86/5 in v3), and introduces the first systematic baseline comparison showing where the skill framework adds measurable value.

Score improvements from v3 to v4. Both below-5 v3 scenarios were fixed via generalised decision-framework updates:

- `flink_micro_commit_scatter`: 3/5 → 5/5 via z-order predicate-type rule (equality-only filters → sort, not z-order) and a corrected fixture.

- **late_arriving_data:** 4/5 $\rightarrow$ 5/5 via Action K prerequisite check (`has_sort_order=true` $\rightarrow$ skip K entirely).

Baseline comparison findings. The skill’s largest lifts occur in *cross-signal* scenarios where the correct action requires combining multiple weak signals from profile, workload, and intent data:

- **late_arriving_data:** baseline 2/5 FAIL $\rightarrow$ skill 5/5 (+3) — the baseline defaults to bin-pack (wrong) and redundantly recommends K (already set). The skill reads `has_sort_order` and the `late_data` flag to prescribe targeted sort compaction.
- **flink_micro_commit_scatter:** baseline 4/5 $\rightarrow$ skill 5/5 (+1) — without workload filter-type analysis, the baseline cannot determine that equality-only predicates make z-order wrong.
- **cdc_high_churn:** baseline 4/5 $\rightarrow$ skill 5/5 (+1) — the COW mode evaluation requires combining write-size and query-rate signals.

For scenarios dominated by a single strong signal (format version, GDPR ordering, count vs time math, z-order collapse), the baseline already reaches 5/5 — the skill adds nothing because Claude’s training data contains the relevant knowledge and the raw profile alone is sufficient.

The 22 scenarios test.

- **Action discrimination.** E1 vs E2 vs L vs Z.
- **Configuration correctness.** Z-order column count, bloom filter placement, file size targets for wide schemas.
- **Ordering requirements.** Compact before expire, expire before orphan cleanup, format-version upgrade before E1.
- **Temporal correctness.** Time-based vs count-based expiry for high-frequency CDC tables.
- **Cost awareness.** Explicit do-nothing when maintenance cost exceeds lifetime query benefit.